SNOWFLAKE DATA ENGINEERING PROJECT

# Healthcare Intelligence Hub

Patient, Cost & Care Analytics

## Phase 3 — Project Design Phase

Institutional template-based phase document

| | |
|---|---|
| Student | Aashi Bindal |
| Enrollment No. | 0827CI241004 |
| Team ID | [Team ID] |
| Institution | Acropolis Institute of Technology and Research · Department of CSIT |
| Faculty Guide | [Faculty Guide] |
| Academic Session | [Academic Session] |
| Document date | October 2026 |

Prepared from the verified project source (query.sql, Health/streamlit_app.py, Health/snowflake.yml, six CSV datasets). Bracketed fields are placeholders for information not supplied.

# Contents


3.1 Problem – Solution Fit ..... 1

3.2 Proposed Solution (Project Proposal) ..... 2

3.3 Solution Architecture ..... 3

3.4 Data Layers and Pipeline Design ..... 6

3.5 Snowflake Database and Schema Design ..... 7

3.6 Fact and Dimension Model ..... 8

3.7 Data Transformation Design ..... 9

3.8 Application and Dashboard Design ..... 10

3.9 Security, Access Control and Data Privacy ..... 10

# Phase 3 — Project Design Phase

**Project:** Healthcare Intelligence Hub: Patient, Cost & Care Analytics **Templates covered:** Problem – Solution Fit (2 marks) · Project Proposal (Proposed Solution) · Solution Architecture (4 marks)

> **Template note.** `Project Proposal (Proposed Solution) template.docx` appears in the Drive folder but was not uploaded. Section 3.2 uses the widely used six-parameter "Proposed Solution" format: problem, idea, novelty, social impact, business model, scalability. Replace it with the official layout if it differs. The diagrams show only objects that exist in `query.sql` or `Health/`, unless a component is labelled *proposed* and drawn dashed.

## 3.1 Problem – Solution Fit

| Field | Value |
|---|---|
| Date | [Submission Date] *(template shows 19 September 2022)* |
| Team ID | [Team ID] |
| Project Name | Healthcare Intelligence Hub: Patient, Cost & Care Analytics |
| Maximum Marks | 2 Marks |

**Problem – Solution Fit Template:** The Problem-Solution Fit simply means that you have found a problem with your customer and that the solution you have realised for it actually solves the customer's problem.

**Purpose:** Solve complex problems in a way that fits the state of your customers. Succeed faster and increase solution adoption by tapping into existing channels of behaviour. Sharpen communication with the right triggers. Understand the existing situation in order to improve it.

*Table 3.1 — Problem–Solution Fit canvas (CC BY-NC-ND 4.0 canvas by Daria Nepriakhina / Amaltama)*

| 1. CUSTOMER SEGMENT(S) — CS | 6. CUSTOMER CONSTRAINTS — CC | 5. AVAILABLE SOLUTIONS — AS |
|---|---|---|
| Hospital administrators and finance teams in a multi-facility hospital network (10 facilities, 60 doctors); secondary: clinical quality leads. | Limited analyst time; no data-engineering staff; data spread across systems; patient privacy obligations; budget for tools. | Monthly spreadsheet exports joined with VLOOKUP; static slide charts; vendor HIS reports. Pros: familiar. Cons: manual, slow, inconsistent, no drill-down. |
| **2. JOBS-TO-BE-DONE / PROBLEMS — J&P** | **9. PROBLEM ROOT CAUSE — RC** | **7. BEHAVIOUR — BE** |
| Get a consolidated, queryable view of visits, spanning cost, LOS, satisfaction and readmissions. Answer: which facilities and doctors are performing well; which diagnoses drive the longest and most expensive stays; where patient satisfaction is at risk. | Visit data sits fragmented across raw operational records, and there is no structured warehouse linking each visit to patient, doctor, facility and diagnosis. KPI definitions are not shared, and labels are inconsistent between sources. | Request extracts from IT; clean data by hand; build pivot tables; email static reports before review meetings. |
| **3. TRIGGERS — TR** | **10. YOUR SOLUTION — SL** | **8. CHANNELS OF BEHAVIOUR — CH** |
| Month-end or quarterly reviews; rising costs; an unexpected readmission spike; board or regulator questions. | A Snowflake pipeline (RAW → STAGING → CURATED → MART) that consolidates six sources into a visit-grain star schema, with ten KPI views and a Streamlit-in-Snowflake dashboard offering six KPIs, eight charts, two tables and five filters. Runs in one platform, needs no credentials in the app, and can be rebuilt by re-running one script. | **8.1 Online:** Snowsight worksheets and the Streamlit app; email for sharing extracts. **8.2 Offline:** review meetings, printed reports, discussions with department heads. |
| **4. EMOTIONS: BEFORE / AFTER — EM** | | |
| **Before:** overwhelmed, unsure of accuracy, reactive. **After:** confident, in control, able to answer follow-up questions live. | | |

## 3.2 Proposed Solution (Project Proposal)

| Field | Value |
|---|---|
| Date | [Submission Date] |
| Team ID | [Team ID] |
| Project Name | Healthcare Intelligence Hub: Patient, Cost & Care Analytics |
| Maximum Marks | [as per template] |

*Table 3.2 — Proposed solution*

| S.No. | Parameter | Description |
|---|---|---|
| 1 | Problem Statement (Problem to be solved) | Hospital administrators and finance teams need a consolidated, queryable view of patient visits — spanning cost, length of stay, satisfaction and readmissions — that today sits fragmented across raw operational records. Without a structured warehouse, it is difficult to answer basic questions such as which facilities and doctors are performing well, which diagnoses drive the longest and most expensive stays, and where patient satisfaction is at risk. |

| S.No. | Parameter | Description |
|---|---|---|
| 2 | Idea / Solution description | Load six CSV sources into Snowflake `HEALTH_DB`; clean them in `STAGING`; model a visit-grain fact table and five dimensions in `CURATED`; encode KPIs in ten `MART` views; deliver an interactive Streamlit-in-Snowflake dashboard with date, facility, specialty, visit-type and diagnosis-category filters. |
| 3 | Novelty / Uniqueness | End-to-end inside one governed platform: storage, transformation, KPI logic and UI. The app authenticates through the active Snowpark session, with no secrets. Every metric traces back through layered schemas. Built-in validation queries run at each layer. |
| 4 | Social Impact / Customer Satisfaction | Faster identification of high-readmission diagnoses (e.g., Depression at 14.05%) and low-satisfaction areas supports better patient care. Transparency on out-of-pocket payment (about 27.7% of total cost) supports fairer financial counselling. |
| 5 | Business Model (Revenue Model) | Internal analytics capability: value comes from reduced reporting effort and better cost and quality decisions. Running cost is Snowflake consumption, which is pay-per-use with auto-suspended warehouses. It could be packaged as a template for other hospital groups. |
| 6 | Scalability of the Solution | Snowflake scales compute (warehouse size, multi-cluster) separately from storage. New facilities or months need only new rows. To scale the app beyond about $10^6$ rows, move filtering and aggregation into SQL (MART views or Snowpark) instead of loading the full fact table into pandas (Phase 8). |

## 3.3 Solution Architecture

| Field | Value |
|---|---|
| Date | 19 September 2026 *(as printed on template)* |
| Team ID | [Team ID] |
| Project Name | Healthcare Intelligence Hub: Patient, Cost & Care Analytics |
| Maximum Marks | 4 Marks |

**Solution Architecture:** Solution architecture bridges the gap between business problems and technology solutions. Its goals are to find the best technical solution, describe the structure and behaviour of the software to stakeholders, define features and phases, and provide specifications for delivery.

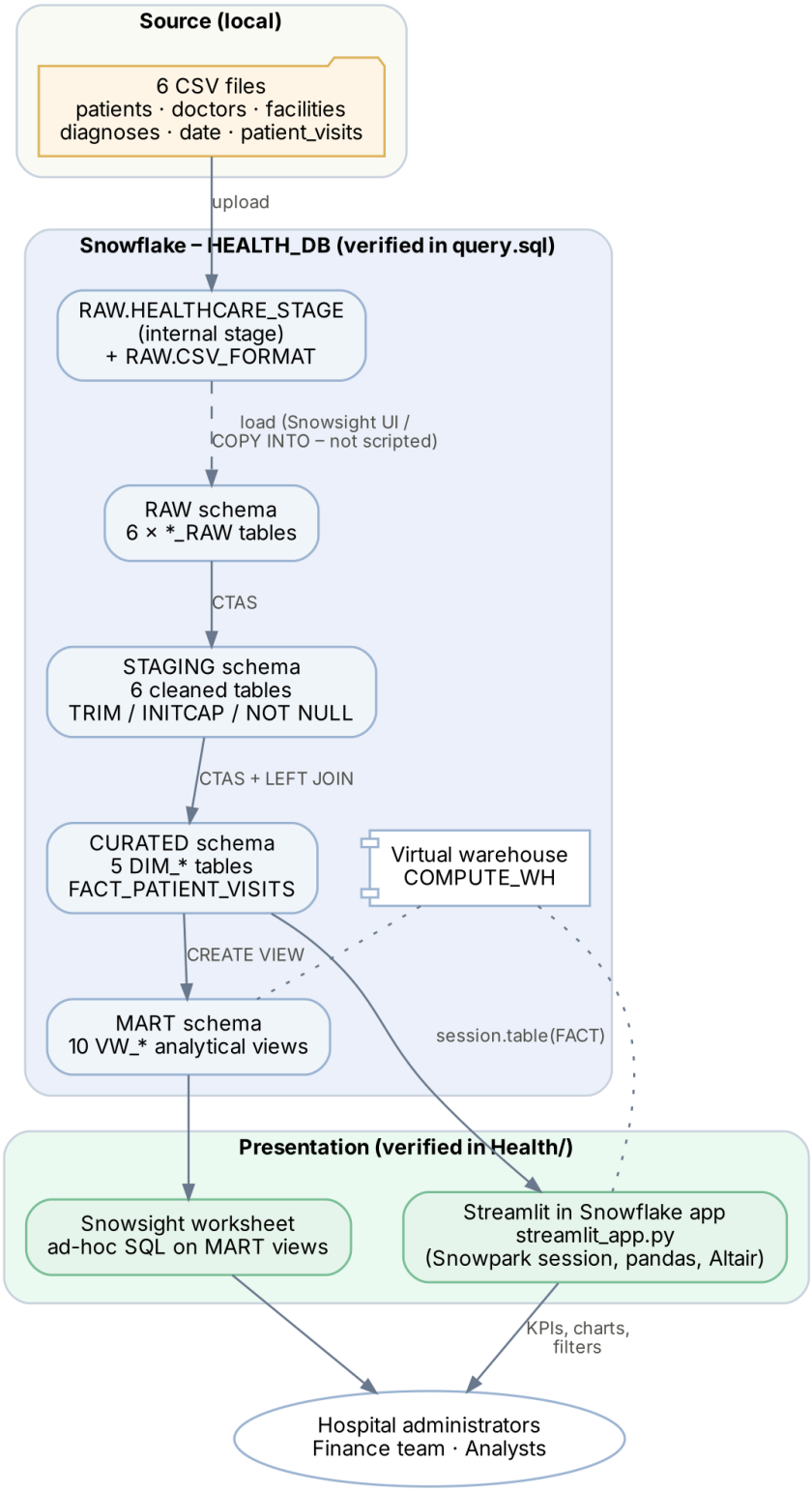



*Figure 3.1 — Solution architecture as implemented. The dashed load arrow marks the one step that is not scripted in the repository.*

**Component Description Table**

*Table 3.3 — Architecture components*

| Component Name | Description / Role in Architecture | Technology Used |
|---|---|---|
| Source CSV files | Six extracts: patients, doctors, facilities, diagnoses, date and patient_visits (about 1.4 MB, 14,059 data rows) | CSV (UTF-8, header row, ISO dates) |
| CSV file format | Parsing rules: `SKIP_HEADER = 1`, `FIELD_OPTIONALLY_ENCLOSED_BY = '"'`, `NULL_IF = ('NULL','null','')`, `EMPTY_FIELD_AS_NULL`, `DATE_FORMAT = 'YYYY-MM-DD'` | Snowflake `FILE FORMAT` `HEALTH_DB.RAW.CSV_FORMAT` |
| Internal stage | Landing area for uploaded files | Snowflake `STAGE` `HEALTH_DB.RAW.HEALTHCARE_STAGE` |
| RAW layer | Typed copies of each source, unmodified | 6 tables `*_RAW` in schema `RAW` |
| STAGING layer | Cleaned and standardised tables; NULL-key rows removed | 6 CTAS tables in schema `STAGING` |
| CURATED layer | Star schema: 5 dimensions + visit fact with denormalised descriptive attributes | `DIM_PATIENT`, `DIM_DOCTOR`, `DIM_FACILITY`, `DIM_DIAGNOSIS`, `DIM_DATE`, `FACT_PATIENT_VISITS` |
| MART layer | Reusable KPI and aggregate logic | 10 views `VW_*` in schema `MART` |
| Virtual warehouse | Compute for SQL and app queries | `COMPUTE_WH` |
| Streamlit app | Interactive dashboard: reads FACT through Snowpark, filters and aggregates in pandas, charts in Altair | Streamlit in Snowflake; `streamlit_app.py`; object `HEALTH_DB.RAW.ICYNJI7QNBXR2IUI`, title "HEALTH" |
| Deployment descriptor | Declares app artefacts, warehouse and compute pool | `snowflake.yml` (definition_version 2), `pyproject.toml` |

## 3.4 Data Layers and Pipeline Design

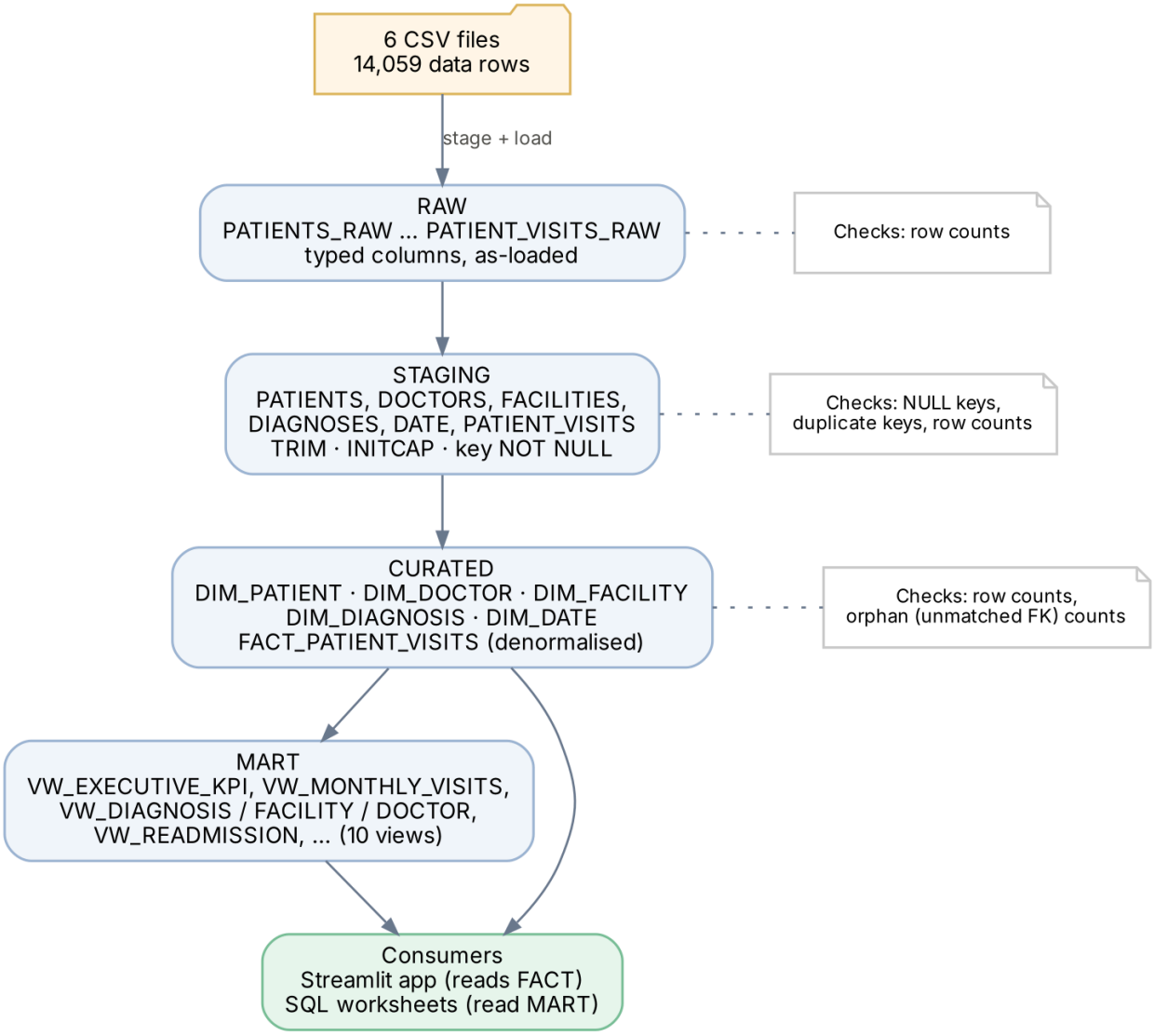



*Figure 3.2 — Layered data pipeline with the validation queries run at each layer*

*Table 3.4 — Layer responsibilities*

| Layer | Schema | Objects | Responsibility | Materialisation |
|---|---|---|---|---|
| Source | — | 6 CSV files | System-of-record extracts | Files |
| Raw | `RAW` | 6 `*_RAW` tables, `CSV_FORMAT`, `HEALTHCARE_STAGE` | Land data with types; no business logic | Tables ( `CREATE OR REPLACE` ) |
| Staging (cleaned) | `STAGING` | `PATIENTS`, `DOCTORS`, `FACILITIES`, `DIAGNOSES`, `DATE`, `PATIENT_VISITS` | Trim, case-standardise, drop NULL keys | CTAS tables |
| Curated (analytical model) | `CURATED` | 5 `DIM_*`, `FACT_PATIENT_VISITS` | Conformed dimensions; visit-grain fact | CTAS tables |
| Mart (consumption) | `MART` | 10 `VW_*` | KPI and aggregate definitions | Views |

## 3.5 Snowflake Database and Schema Design

```
HEALTH_DB
├── RAW       PATIENTS_RAW, DOCTORS_RAW, FACILITIES_RAW, DIAGNOSES_RAW, DATE_RAW,
│             PATIENT_VISITS_RAW, CSV_FORMAT (file format), HEALTHCARE_STAGE (stage),
│             ICYNJI7QNBXR2IUI (Streamlit app, per snowflake.yml)
├── STAGING   PATIENTS, DOCTORS, FACILITIES, DIAGNOSES, DATE, PATIENT_VISITS
├── CURATED   DIM_PATIENT, DIM_DOCTOR, DIM_FACILITY, DIM_DIAGNOSIS, DIM_DATE,
│             FACT_PATIENT_VISITS
└── MART      VW_EXECUTIVE_KPI, VW_MONTHLY_VISITS, VW_DIAGNOSIS_ANALYTICS,
              VW_FACILITY_ANALYTICS, VW_VISIT_TYPE_DISTRIBUTION, VW_TREATMENT_ANALYTICS,
              VW_ADMISSION_STATUS_ANALYTICS, VW_PATIENT_VISIT_TYPE_ANALYSIS,
              VW_DOCTOR_ANALYTICS, VW_READMISSION_ANALYTICS
```

**Design observation:** the Streamlit app object is created in the `RAW` schema (per `snowflake.yml`). A dedicated `APP` or `MART` schema would keep the raw layer free of consumption objects and simplify grants.

## 3.6 Fact and Dimension Model

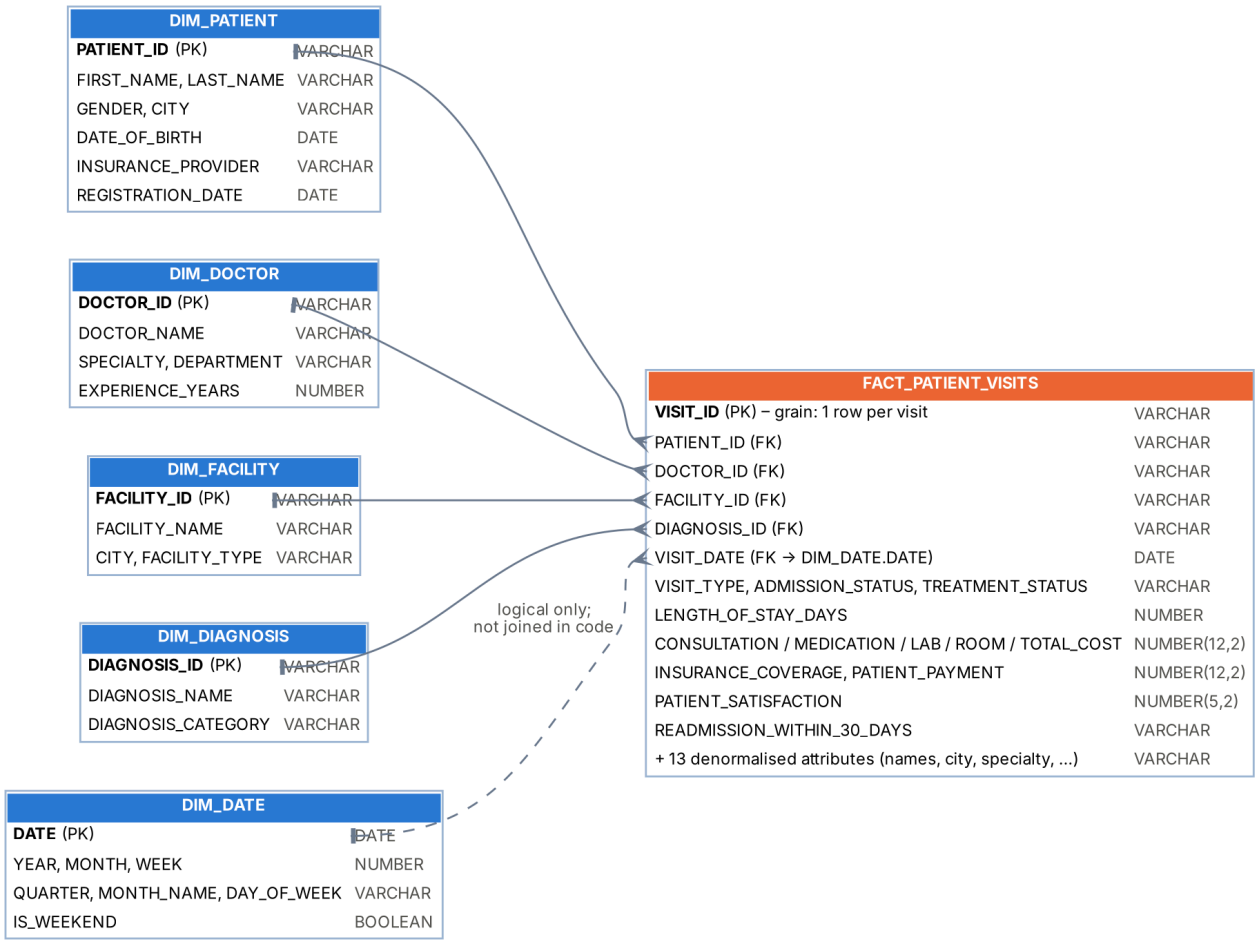



*Figure 3.3 — ER diagram of the CURATED star schema (keys are logical: no PK/FK constraints are declared in query.sql)*

*Table 3.5 — Keys, grain and relationships*

| Table | Grain | Primary key (logical) | Foreign keys | Rows | Relationship to fact |
|---|---|---|---|---|---|
| `FACT_PATIENT_VISITS` | One row per patient visit | `VISIT_ID` | `PATIENT_ID`, `DOCTOR_ID`, `FACILITY_ID`, `DIAGNOSIS_ID`, `VISIT_DATE` | 10,000 | — |
| `DIM_PATIENT` | One row per patient | `PATIENT_ID` | — | 3,000 | 1 : N (2,889 patients have ≥1 visit) |
| `DIM_DOCTOR` | One row per doctor | `DOCTOR_ID` | — | 60 | 1 : N |
| `DIM_FACILITY` | One row per facility | `FACILITY_ID` | — | 10 | 1 : N |
| `DIM_DIAGNOSIS` | One row per diagnosis | `DIAGNOSIS_ID` | — | 15 | 1 : N |
| `DIM_DATE` | One row per calendar day | `DATE` | — | 974 | 1 : N via `VISIT_DATE` (logical only) |

**Model characteristics (verified from code):**

1. **Denormalised fact.** `FACT_PATIENT_VISITS` carries 13 descriptive attributes copied from the dimensions (patient name, gender, city, insurer, doctor name, specialty, department, facility name, city and type, diagnosis name and category). This lets the app and views read one table with no joins. The cost is redundancy, and dimension changes require a fact rebuild.
2. **DIM_DATE is built but unused.** No view or app code joins to it. Monthly grouping uses `DATE_TRUNC` on `VISIT_DATE`, so weekday and weekend attributes are not yet exploited.
3. **No declared constraints.** Snowflake records but does not enforce PK and FK constraints. Declaring them (`ALTER TABLE … ADD PRIMARY KEY`) would still help documentation and BI tools.
4. **Dimensions are rebuilt in full** (`CREATE OR REPLACE`, Type 1). There is no history (SCD Type 2).

## 3.7 Data Transformation Design

*Table 3.6 — Transformation rules by layer*

| Step | Source → Target | Rule | Columns |
|---|---|---|---|
| T1 | CSV → `*_RAW` | Parse with `CSV_FORMAT`; empty and 'NULL' strings become NULL; dates parsed from `YYYY-MM-DD` | all |
| T2 | `*_RAW` → `STAGING.*` | `TRIM()` on every text column | IDs, names, categorical values |
| T3 | `*_RAW` → `STAGING.*` | `INITCAP(TRIM())` on categorical columns | GENDER, CITY, SPECIALTY, DEPARTMENT, FACILITY_TYPE, DIAGNOSIS_CATEGORY, VISIT_TYPE, ADMISSION_STATUS, TREATMENT_STATUS, READMISSION_WITHIN_30_DAYS |
| T4 | `*_RAW` → `STAGING.*` | `WHERE <key> IS NOT NULL` | each business key |
| T5 | `STAGING.*` → `DIM_*` | `SELECT DISTINCT` (removes exact duplicate rows) | all |
| T6 | `STAGING.*` → `FACT_PATIENT_VISITS` | `LEFT JOIN` visits to patients, doctors, facilities and diagnoses on their IDs; rename `CITY` to `PATIENT_CITY` / `FACILITY_CITY` | 19 visit columns + 13 attributes |
| T7 | `FACT` → `MART.VW_*` | `COUNT`, `COUNT DISTINCT`, `SUM`, `AVG`, `ROUND(…,2)`, `CASE WHEN flag='Yes'`, `DATE_TRUNC('MONTH')`, `GROUP BY`, `ORDER BY` | measures |

**Design risks found during verification (details and evidence in Phase 6):**

- T3 `INITCAP` lower-cases letters after the first and treats `-` as a word break. Snowflake will therefore produce `Ent` (from `ENT`) and `Follow-Up` (from `Follow-up`). Fix: exclude acronyms, or apply `INITCAP` only where the source is inconsistent.
- `VW_DOCTOR_ANALYTICS` and the app leaderboard group by `DOCTOR_NAME, SPECIALTY, DEPARTMENT`. Doctors D018 and D020 share all three values ("Dr. Ananya Reddy", Orthopedics, Specialty Care), so they merge into one row: 59 rows for 60 doctors. Fix: add `DOCTOR_ID` to the grouping.
- `VW_VISIT_TYPE_DISTRIBUTION` and `VW_PATIENT_VISIT_TYPE_ANALYSIS` return the same columns and differ only in sort order (redundant).

## 3.8 Application and Dashboard Design

The app is a single Streamlit page ( `layout="wide"` , sidebar expanded) built in `streamlit_app.py` .

*Table 3.7 — Dashboard layout (wireframe of the implemented page)*

<table>
<tr><td rowspan="8">Sidebar: Filters<br>Date Range (date_input)<br>Facility (selectbox)<br>Doctor Specialty<br>Visit Type<br>Diagnosis Category<br>each defaults to "All"</td><td colspan="2">Header: "Healthcare Intelligence Hub", caption "Patient, Cost & Care Analytics"</td></tr>
<tr><td colspan="2">KPI row (6 × st.metric): Total Visits · Unique Patients · Total Cost · Avg Satisfaction (/5) · Readmission Rate (%) · Avg Length of Stay (days)</td></tr>
<tr><td>Monthly Visit Trend: line + points (Altair)</td><td>Visit Type Split: donut</td></tr>
<tr><td>Top 10 Diagnoses by Volume: horizontal bar</td><td>Admission Status: donut</td></tr>
<tr><td>Facility Performance: bar coloured by facility type</td><td>Facility table: visits, avg cost, avg satisfaction</td></tr>
<tr><td>Treatment Status: bar</td><td>Readmission Rate by Diagnosis: top-10 horizontal bar</td></tr>
<tr><td colspan="2">Average Cost Breakdown by Visit Type: stacked bar (consultation, medication, lab, room)</td></tr>
<tr><td colspan="2">Doctor Performance Leaderboard: table · footer: filtered visit count and source table</td></tr>
</table>

**Data access design:** `load_fact()` reads `HEALTH_DB.CURATED.FACT_PATIENT_VISITS` once through `session.table(...).to_pandas()` , upper-cases column names, converts `VISIT_DATE` to datetime, and is cached for 600 seconds. All filtering and aggregation then happen in pandas. The MART views are **not** used by the app. They serve SQL users and the documentation of KPI logic.

## 3.9 Security, Access Control and Data Privacy

**Implemented:** - No credentials in code. The app uses `get_active_session()` , which runs with the owner's or caller's Snowflake role, according to the Streamlit-in-Snowflake model. - Data stays inside Snowflake. There are no external APIs or network calls.

**Not implemented (gaps):** - No `CREATE ROLE` or `GRANT` statements in `query.sql` ; development presumably used a high-privilege role. - Patient first and last names and date of birth are carried into the fact table and app data frame. The data appears synthetic, but a real deployment would need protection.

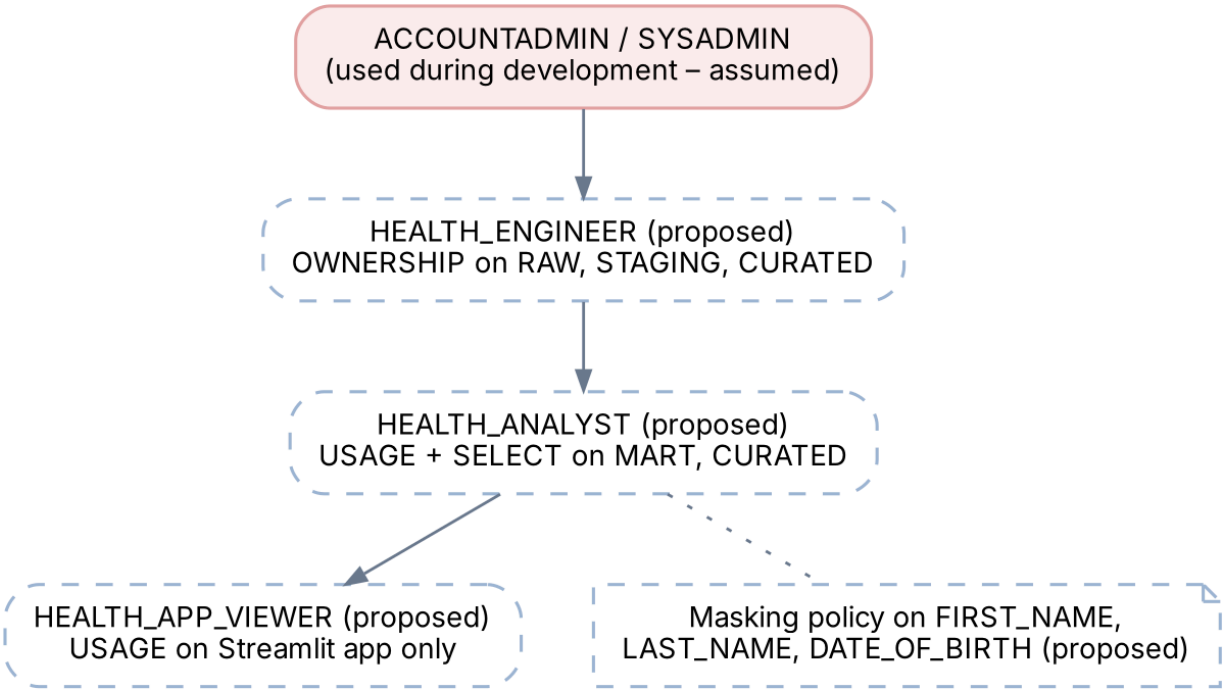



*Figure 3.4 — Proposed role hierarchy and masking (dashed = proposed, not implemented)*

**Proposed controls:** least-privilege roles ( `HEALTH_ENGINEER` , `HEALTH_ANALYST` , `HEALTH_APP_VIEWER` ); `SELECT` on `MART` only for analysts; dynamic data-masking policies on `FIRST_NAME` , `LAST_NAME` and `DATE_OF_BIRTH` ; dropping patient names from `FACT_PATIENT_VISITS` because no visual uses them; moving the app out of `RAW` . A real deployment must also comply with applicable health-data regulations (e.g., India's Digital Personal Data Protection Act, 2023, or HIPAA in the US).